SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 11.2

Program	**: B. Tech (CSE)**
Specialization	**: -**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: II**
Academic Session	**: 2025-2026**
Name of Student	**: K.Pavan**
Enrollment No.	**: 2403A51L19**
Batch No.	**: 51**
Date	**: 13/03/26**

Submission Starts here

Screenshots:

Task-1:
Prompt:

Create a Python class implementing a Stack data structure.

Requirements:
- Use a list internally.
- Methods: push(), pop(), peek(), is_empty(), size().
- Follow LIFO behavior.
- Handle pop from empty stack safely.
- Add docstrings, comments, and a small example showing stack operations.

SR UNIVERSITY

Code:

```python
class Stack:
    """
    A simple Stack (LIFO) data structure implementation using a Python list.
    """

    def __init__(self):
        """
        Initialize an empty stack.
        """
        # Internal list to store stack elements
        self._items = []

    def push(self, item):
        """
        Push an item onto the top of the stack.

        :param item: The item to be added to the stack.
        """
        self._items.append(item)

    def pop(self):
        """
        Remove and return the top item of the stack.

        Handles popping from an empty stack safely by returning None.

        :return: The top item if the stack is not empty, otherwise None.
        """
        if self.is_empty():
            # Safely handle empty stack: return None instead of raising an error
            return None
        return self._items.pop()

    def peek(self):
        """
        Return the top item of the stack without removing it.

        :return: The top item if the stack is not empty, otherwise None.
        """
        if self.is_empty():
            return None
        return self._items[-1]

    def is_empty(self):
        """
        Check whether the stack is empty.

        :return: True if the stack has no elements, False otherwise.
        """
        return len(self._items) == 0

    def size(self):
        """
        Return the number of items currently in the stack.

        :return: Integer size of the stack.
        """
        return len(self._items)


```

```python
if __name__ == "__main__":
    # Small example showing stack operations

    stack = Stack()

    print("Is stack empty?", stack.is_empty())  # True

    # Push some items
    stack.push(10)
    stack.push(20)
    stack.push(30)

    print("Current size:", stack.size())        # 3
    print("Top element (peek):", stack.peek())  # 30

    # Pop elements
    print("Popped:", stack.pop())               # 30
    print("Popped:", stack.pop())               # 20

    print("Current size:", stack.size())        # 1
    print("Top element (peek):", stack.peek())  # 10

    # Pop remaining element
    print("Popped:", stack.pop())               # 10

    # Safely pop from empty stack
    print("Popped from empty stack:", stack.pop())  # None
    print("Is stack empty now?", stack.is_empty())  # True


```

Output:

```
Is stack empty? True
Current size: 3
Top element (peek): 30
Popped: 30
Popped: 20
Current size: 1
Top element (peek): 10
Popped: 10
Popped from empty stack: None
Is stack empty now? True
```

TASK -2:

Prompt:

Write a Python program implementing a Queue class.

Requirements:
- Follow FIFO principle.
- Methods: enqueue(), dequeue(), front(), is_empty(), size().
- Handle empty queue cases.
- Add comments, docstrings, and a short demo of queue operations.

Code:

```python
class Queue:
    """
    A simple FIFO (First-In-First-Out) queue implementation.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self._items = []

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
            bool: True if the queue has no elements, False otherwise.
        """
        return len(self._items) == 0

    def enqueue(self, item):
        """
        Add an item to the back of the queue.

        Args:
            item: The element to be added.
        """
        # Insert at index 0, so the "front" is at the end of the list.
        self._items.insert(0, item)

    def dequeue(self):
        """
        Remove and return the item from the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        # Pop from the end, which represents the front of the queue.
        return self._items.pop()

    def front(self):
        """
        Return (without removing) the item at the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("front from empty queue")
        return self._items[-1]

    def size(self):
        """
        Return the number of items in the queue.

        Returns:
            int: Number of elements currently in the queue.
        """
        return len(self._items)
```

```python

def demo_queue_operations():
    """
    Demonstrate basic operations of the Queue class.
    """
    q = Queue()

    print("Is queue empty?", q.is_empty())
    print("Enqueueing 10, 20, 30...")
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)

    print("Current size:", q.size())
    print("Front element:", q.front())  # Should be 10 (first in)

    print("Dequeued:", q.dequeue())     # Removes 10
    print("Dequeued:", q.dequeue())     # Removes 20
    print("Current front:", q.front())  # Should be 30
    print("Current size:", q.size())
    print("Is queue empty?", q.is_empty())

    # Demonstrate handling of empty queue
    print("\nClearing remaining elements...")
    q.dequeue()  # remove 30

    print("Is queue empty now?", q.is_empty())
    try:
        q.dequeue()
    except IndexError as e:
        print("Error when dequeuing from empty queue:", e)

    try:
        q.front()
    except IndexError as e:
        print("Error when getting front from empty queue:", e)


if __name__ == "__main__":
    demo_queue_operations()
```

Output:

```
Is queue empty? True
Enqueueing 10, 20, 30...
Is queue empty? True
Enqueueing 10, 20, 30...
Enqueueing 10, 20, 30...
Current size: 3
Front element: 10
Dequeued: 10
Current size: 3
Front element: 10
Dequeued: 10
Dequeued: 10
Dequeued: 20
Current front: 30
Dequeued: 20
Current front: 30
Current front: 30
Current size: 1
Is queue empty? False
Current size: 1
Is queue empty? False

Is queue empty? False


Clearing remaining elements...
Is queue empty now? True
Error when dequeuing from empty queue: dequeue from empty queue
Error when getting front from empty queue: front from empty queue
Error when getting front from empty queue: front from empty queue
```

TASK-3:

Prompt:

Create a Python implementation of a Singly Linked List.

Requirements:
- Node class with data and next pointer.
- LinkedList class with insert_at_beginning(), insert_at_end(), and display().
- Show traversal of the list.

- Include comments, docstrings, and a simple usage example.

Code:

SR UNIVERSITY

```python
"""

This module defines:
- Node: a single node in a singly linked list.
- LinkedList: a singly linked list with basic insertion and display operations.

It also contains a simple usage example that demonstrates:
- Inserting nodes at the beginning and at the end.
- Traversing and displaying the list contents.
"""

from __future__ import annotations
from typing import Any, Optional


class Node:
    """
    Represents a single node in a singly linked list.

    Attributes
    ----------
    data : Any
        The value stored in the node.
    next : Optional[Node]
        Reference to the next node in the list, or None if this is the last node.
    """

    def __init__(self, data: Any) -> None:
        """
        Initialize a new node with the given data.

        Parameters
        ----------
        data : Any
            The value to store in the node.
        """
        self.data: Any = data
        self.next: Optional[Node] = None

    def __repr__(self) -> str:
        """Return a string representation of the node."""
        return f"Node({self.data!r})"


class LinkedList:
    """
    A simple singly linked list implementation.

    The list maintains a reference to the head node. New nodes can be
    inserted at the beginning or at the end, and the list can be traversed
    to display all elements.
    """

    def __init__(self) -> None:
        """Initialize an empty linked list."""
        self.head: Optional[Node] = None

    def insert_at_beginning(self, data: Any) -> None:
        """
        Insert a new node containing `data` at the beginning of the list.

        This operation runs in O(1) time.
        """
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```python

    def insert_at_end(self, data: Any) -> None:
        """
        Insert a new node containing `data` at the end of the list.

        This operation runs in O(n) time because it traverses the list
        to find the last node.
        """
        new_node = Node(data)

        # If the list is empty, the new node becomes the head.
        if self.head is None:
            self.head = new_node
            return

        # Otherwise, traverse to the last node.
        current = self.head
        while current.next is not None:
            current = current.next

        # Link the last node to the new node.
        current.next = new_node

    def display(self) -> None:
        """
        Traverse the list and print each node's data.

        This demonstrates list traversal from head to tail.
        """
        current = self.head
        elements = []

        # Traverse the list, collecting values.
        while current is not None:
            elements.append(str(current.data))
            current = current.next

        # Join collected values with arrows to visualize the links.
        print(" -> ".join(elements) if elements else "Empty list")


if __name__ == "__main__":
    # Simple usage example for the Singly Linked List.

    # Create an empty linked list.
    linked_list = LinkedList()

    # Insert elements at the beginning.
    linked_list.insert_at_beginning(30)
    linked_list.insert_at_beginning(20)
    linked_list.insert_at_beginning(10)

    # Insert elements at the end.
    linked_list.insert_at_end(40)
    linked_list.insert_at_end(50)

    # Display/traverse the list.
    # Expected output: 10 -> 20 -> 30 -> 40 -> 50
    print("Contents of the linked list:")
    linked_list.display()

```

Output:

```
Contents of the linked list:
10 -> 20 -> 30 -> 40 -> 50
```

TASK-4:

Prompt:
Implement a Binary Search Tree in Python.

Requirements:
- Node class with value, left, and right.
- Methods: insert() and inorder_traversal().
- Follow BST rules (left < root < right).
- Add comments, docstrings, and example insertions with traversal output.

Code:

```python
class Node:
    """
    A node in a Binary Search Tree (BST).

    Attributes
    ----------
    value : int | float
        The value stored in the node.
    left : Node | None
        The left child node (stores smaller values).
    right : Node | None
        The right child node (stores larger values).
    """

    def __init__(self, value):
        """Initialize a new node with the given value."""
        self.value = value
        self.left = None
        self.right = None


class BinarySearchTree:
    """
    Simple Binary Search Tree (BST) implementation.

    The BST property is:
    - All values in the left subtree  < node.value
    - All values in the right subtree > node.value
    """

    def __init__(self):
        """Create an empty Binary Search Tree."""
        self.root = None

    def insert(self, value):
        """
        Insert a new value into the BST.

        Parameters
        ----------
        value : int | float
            The value to be inserted.
        """
        if self.root is None:
            # Tree is empty: the new node becomes the root.
            self.root = Node(value)
        else:
            # Delegate to the recursive helper starting from the root.
            self._insert_recursive(self.root, value)

    def _insert_recursive(self, current_node, value):
        """
        Recursively insert a value starting from the given node.

        This helper enforces the BST rule: left < root < right.
        Duplicate values are ignored (no insertion) in this implementation.
        """
        if value < current_node.value:
            # Go to the left subtree.
            if current_node.left is None:
                current_node.left = Node(value)
            else:
                self._insert_recursive(current_node.left, value)
        elif value > current_node.value:
            # Go to the right subtree.
```

```python
            if current_node.right is None:
                current_node.right = Node(value)
            else:
                self._insert_recursive(current_node.right, value)
        else:
            # Value already exists in the tree; do nothing to avoid duplicates.
            pass

    def inorder_traversal(self):
        """
        Perform an in-order traversal of the BST.

        Returns
        -------
        list[int | float]
            A list of values in ascending (sorted) order.
        """
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        """
        Recursive helper to perform in-order traversal.

        Visit left subtree, then current node, then right subtree.
        """
        if node is None:
            return
        # Traverse left subtree
        self._inorder_recursive(node.left, result)
        # Visit current node
        result.append(node.value)
        # Traverse right subtree
        self._inorder_recursive(node.right, result)


if __name__ == "__main__":
    # Example usage of the BinarySearchTree

    # Create a BST instance
    bst = BinarySearchTree()

    # Insert some example values
    values_to_insert = [50, 30, 70, 20, 40, 60, 80]
    for v in values_to_insert:
        bst.insert(v)

    # Perform an in-order traversal (should print sorted values)
    traversal_result = bst.inorder_traversal()
    print("In-order traversal of BST:", traversal_result)
```

Output:

```
In-order traversal of BST: [20, 30, 40, 50, 60, 70, 80]
In-order traversal of BST: [20, 30, 40, 50, 60, 70, 80]
```

TASK-5:

Prompt:
Create a Python Hash Table implementation.

Requirements:
- Use an array for storage.
- Handle collisions using chaining (lists).
- Methods: insert(key,value), search(key), delete(key).
- Use hash(key) % table_size.
- Add comments, docstrings, and a small test example.

Code:

```python
class HashTable:
    """
    A simple hash table implementation using:
    - Python's built-in hash()
    - Separate chaining with lists to handle collisions
    """

    def __init__(self, table_size=10):
        """
        Initialize the hash table.

        Args:
            table_size (int): Number of buckets in the underlying array.
        """
        self.table_size = table_size
        # Each bucket will hold a list of (key, value) pairs (for chaining)
        self.table = [[] for _ in range(table_size)]

    def _hash(self, key):
        """
        Compute the array index for a given key using hash(key) % table_size.

        Args:
            key: The key to hash.

        Returns:
            int: The index in the table array.
        """
        return hash(key) % self.table_size

    def insert(self, key, value):
        """
        Insert or update a key-value pair in the hash table.

        If the key already exists, its value is updated.

        Args:
            key: The key to insert.
            value: The value associated with the key.
        """
        index = self._hash(key)
        bucket = self.table[index]

        # Check if key already exists in the bucket; if so, update it
        for i, (k, v) in enumerate[Any](bucket):
            if k == key:
                bucket[i] = (key, value)
                return

        # If key not found, append new (key, value) pair
        bucket.append((key, value))

    def search(self, key):
        """
        Search for a key in the hash table.

        Args:
            key: The key to search for.

        Returns:
            The associated value if found, otherwise None.
        """
        index = self._hash(key)
        bucket = self.table[index]

        for k, v in bucket:
            if k == key:
```

```python
                return v

        return None

    def delete(self, key):
        """
        Delete a key-value pair from the hash table if it exists.

        Args:
            key: The key to delete.

        Returns:
            bool: True if the key was found and deleted, False otherwise.
        """
        index = self._hash(key)
        bucket = self.table[index]

        for i, (k, v) in enumerate[Any](bucket):
            if k == key:
                # Remove the (key, value) pair from the bucket
                del bucket[i]
                return True

        return False


if __name__ == "__main__":
    # Small test example

    # Create a hash table with 5 buckets
    ht = HashTable(table_size=5)

    # Insert some key-value pairs
    ht.insert("apple", 10)
    ht.insert("banana", 20)
    ht.insert("orange", 30)

    # Update an existing key
    ht.insert("banana", 25)

    # Search for keys
    print("apple:", ht.search("apple"))     # Expected: 10
    print("banana:", ht.search("banana"))   # Expected: 25
    print("orange:", ht.search("orange"))   # Expected: 30
    print("grape:", ht.search("grape"))     # Expected: None (not found)

    # Delete a key
    print("Delete 'banana':", ht.delete("banana"))  # Expected: True
    print("banana after delete:", ht.search("banana"))  # Expected: None

    # Attempt to delete a non-existing key
    print("Delete 'grape':", ht.delete("grape"))    # Expected: False
```

Output:

```
apple: 10
apple: 10
banana: 25
banana: 25
orange: 30
grape: None
grape: None
Delete 'banana': True
banana after delete: None
Delete 'grape': False
banana after delete: None
Delete 'grape': False
Delete 'grape': False
```